

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序序设计

批改人：

实验	CUDA 并行矩阵乘法	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/6/25

1. 实验目的

练习 CUDA 编程，学习 CUDA 编程的原理以及 GPU 的架构对编程性能的影响。

2. 实验过程 and 核心代码

我实现了两个版本的 cuda 代码，一个是直接从全局内存中进行数据读取，另一个是利用共享内存来进行优化。每个线程块（Block）都有一个共享内存，一个线程块中的线程可以共享访问它，相比于访问全局内存，访问共享内存无疑是更快的选择。
对于全局内存实现比较简单，每个线程负责一个结果矩阵中元素的计算。

```
__global__ void gemm_naive(const float* A, const float* B, float* C, int m, int n, int k) {
    // 计算当前线程负责的 c 矩阵元素的全局行和列
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    // 边界检查，防止线程访问越界内存
    if (row < m && col < k) {
        float sum = 0.0f;
        // 计算点积：A的第row行 * B的第col列
        // 这里的每一次 A[...] 和 B[...] 都是一次全局内存读取
        for (int i = 0; i < n; ++i) {
            sum += A[row * n + i] * B[i * k + col];
        }
        C[row * k + col] = sum;
    }
}
```

对于共享内存的实现，和上一个实验一样，将矩阵分成 TILE_SIZE x TILE_SIZE 的子矩阵，每个线程负责 C 中一个元素的计算，为了实现这一点，需要不断把不同的 tile 块加载进 Block 里面，每次迭代从全局内存加载一个小的子矩阵（即 tile）到共享内存中。这包括 A 矩阵的一个子矩阵和 B 矩阵的一个子矩阵（A 和 B 分别进行行遍历和列遍历），然后执行一次局部求和，把结果加进 C 中后继续进行下一轮迭代，知道全部遍历完成。

```

__global__ void gemm_shared(const float* A, const float* B, float* C, int m, int n, int k) {
    // 声明共享内存来存放 A 和 B 的子矩阵 (Tile)
    // 每个线程块(Block)都拥有自己的一份共享内存
    __shared__ float tile_A[TILE_SIZE][TILE_SIZE];
    __shared__ float tile_B[TILE_SIZE][TILE_SIZE];

    // 计算当前线程在线程块内的局部索引 (0 到 TILE_SIZE-1)
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // 计算当前线程负责的 C 矩阵元素的全局行和列
    int row = blockIdx.y * TILE_SIZE + ty;
    int col = blockIdx.x * TILE_SIZE + tx;

    float sum = 0.0f;

    // 循环遍历所有需要的子矩阵块来计算最终的点积
    for (int t = 0; t < (n + TILE_SIZE - 1) / TILE_SIZE; ++t) {

        // --- 协作加载数据到共享内存 ---
        // 每个线程负责从全局内存加载一个元素到共享内存
        const int A_row = row;
        const int A_col = t * TILE_SIZE + tx;
        if (A_row < m && A_col < n) {
            tile_A[ty][tx] = A[A_row * n + A_col];
        } else {
            tile_A[ty][tx] = 0.0f;
        }
    }
}

```

```

const int B_row = t * TILE_SIZE + ty;
const int B_col = col;
if (B_row < n && B_col < k) {
    tile_B[ty][tx] = B[B_row * k + B_col];
} else {
    tile_B[ty][tx] = 0.0f;
}

// --- 同步线程块内的所有线程 ---
// 保证所有线程都完成了从全局内存到共享内存的加载后，再进行下一步计算
__syncthreads();

// --- 从高速的共享内存中读取数据，进行计算 ---
// 每个线程计算 TILE_SIZE 次乘加操作
for (int i = 0; i < TILE_SIZE; ++i) {
    sum += tile_A[ty][i] * tile_B[i][tx];
}

// --- 再次同步 ---
// 保证所有线程都完成了本次子块的计算后，再进入下一次循环去加载新的子块
__syncthreads();
}

// 将最终结果写回全局内存
if (row < m && col < k) {
    C[row * k + col] = sum;
}
}

```

3. 实验结果

运行结果如下：

验证运行正确性:

```
ehpc@67a2e57c706a: ~/Desktop$ nvcc mm.cu -o mm
ehpc@67a2e57c706a: ~/Desktop$ ./mm 4 4 4
正在计算矩阵乘法 C(4x4) = A(4x4) * B(4x4)
Naive Kernel 执行时间: 0.140992 ms
Naive Kernel 性能: 0.000907853 GFLOPS

Shared Memory Kernel 执行时间: 0.021184 ms
Shared Memory Kernel 性能: 0.0060423 GFLOPS

Matrix A:
0.397023 0.104507 0.262174 0.0536729
0.633124 0.632679 0.225219 0.410785
0.334174 0.632724 0.317583 0.0763602
0.23567 0.26563 0.950917 0.0893805

Matrix B:
0.942198 0.528812 0.285071 0.447674
0.158186 0.30413 0.101285 0.8211
0.710698 0.658821 0.9436 0.593957
0.109226 0.387014 0.118917 0.506249

Matrix C (Shared Memory):
0.582795 0.435232 0.377535 0.44644
0.901541 0.83458 0.505932 1.14466
0.648993 0.607929 0.4681 0.896418
0.949644 0.866487 1.002 0.933665
```

矩阵规模为 128 时:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 128 128 128
正在计算矩阵乘法 C(128x128) = A(128x128) * B(128x128)
Naive Kernel 执行时间: 0.193856 ms
Naive Kernel 性能: 21.6362 GFLOPS

Shared Memory Kernel 执行时间: 0.044416 ms
Shared Memory Kernel 性能: 94.4323 GFLOPS
```

256:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 256 256 256
正在计算矩阵乘法 C(256x256) = A(256x256) * B(256x256)
Naive Kernel 执行时间: 0.607808 ms
Naive Kernel 性能: 55.2056 GFLOPS

Shared Memory Kernel 执行时间: 0.2096 ms
Shared Memory Kernel 性能: 160.088 GFLOPS
```

512:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 512 512 512
正在计算矩阵乘法 C(512x512) = A(512x512) * B(512x512)
Naive Kernel 执行时间: 3.83002 ms
Naive Kernel 性能: 70.0873 GFLOPS

Shared Memory Kernel 执行时间: 1.38138 ms
Shared Memory Kernel 性能: 194.325 GFLOPS
```

1024:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 1024 1024 1024
正在计算矩阵乘法 C(1024x1024) = A(1024x1024) * B(1024x1024)
Naive Kernel 执行时间: 29.561 ms
Naive Kernel 性能: 72.6458 GFLOPS

Shared Memory Kernel 执行时间: 10.6839 ms
Shared Memory Kernel 性能: 201.002 GFLOPS
```

2048:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 2048 2048 2048
正在计算矩阵乘法 C(2048x2048) = A(2048x2048) * B(2048x2048)
Naive Kernel 执行时间: 220.623 ms
Naive Kernel 性能: 77.8698 GFLOPS

Shared Memory Kernel 执行时间: 70.3603 ms
Shared Memory Kernel 性能: 244.17 GFLOPS
```

从上面的运行结果可以很明显的看出来，矩阵规模越大，运行时间越长。利用共享内存的矩阵乘法的性能几乎是使用全局内存的 3 倍。共享内存的使用大大提高了运行效率。

以上结果都是在 `block_size = 16 * 16` 的（`block_size` 大小即为 `tile_size` 大小）。接下来让我将这个值调大成为 `32 * 32`:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 2048 2048 2048
正在计算矩阵乘法 C(2048x2048) = A(2048x2048) * B(2048x2048)
Naive Kernel 执行时间: 221.798 ms
Naive Kernel 性能: 77.4572 GFLOPS

Shared Memory Kernel 执行时间: 53.0325 ms
Shared Memory Kernel 性能: 323.95 GFLOPS
```

调成 `64 * 64`:

```
ehpc@67a2e57c706a: ~/Desktop$ ./mm 2048 2048 2048
正在计算矩阵乘法 C(2048x2048) = A(2048x2048) * B(2048x2048)
Naive Kernel 执行时间: 234.631 ms
Naive Kernel 性能: 73.2208 GFLOPS

Shared Memory Kernel 执行时间: 0.003232 ms
Shared Memory Kernel 性能: 5.31555e+06 GFLOPS
```

可以看到，随着 `tilesize` 的增加（在我的代码里面 `blocksize=tilesize`），对于全局变量来说几乎没什么影响，但对于共享内存的影响巨大，随着 `tilesize` 的增大，共享内存版本的性能得到巨大提升。分析原因，是因为目前是每个线程对于一个元素的计算需要按行（列）遍历与其有关行（列）的 `tile`，当 `tile` 过于小时，把 `tile` 从全局内存加载到共享内存需要频繁多次，这严重导致性能浪费，（类比

全局内存版本就是可以看成 $\text{tilesize}=1$ 的情况下），而大 tile 可以一次性加载更多的数据到共享内存，从而更好利用共享内存的优势，大大提升性能。

4. 实验感想

通过这次实验，我在 cuda 上实现了并行矩阵乘法的计算，并实现了全局内存和共享内存两种方式的加速方案。并且对共享内存实现的特性进行了详细分析，分析了 tilesize 的大小对性能的影响。通过这次实验，我对 cuda 编程的理解更进一步，受益匪浅。